

Reviewer Pack — Asymptotic Dimension of KW Space for PARITY Matches Optimal ...

Entry b0765766a778 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 03:25:54 UTC

Asymptotic Dimension of KW Space for PARITY Matches Optimal Formula Depth

Entry ID: b0765766a778 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 03:25:54 UTC

1. Conjecture

Field A (mathematical branch): Coarse-Geometric KW Bounds (CG-KW); coarse geometry of metric spaces and asymptotic dimension theory (Gromov, Yu, Nowak-Yu) **Field B** (complexity object): De Morgan formula depth $D(f)$ for f in $\{0,1\}^n \rightarrow \{0,1\}$; the NC^1 / formula-size complexity class measured by the Karchmer-Wigderson communication game

Statement:

For the PARITY function PAR_n on n bits, the asymptotic dimension of the KW metric space satisfies $\text{asdim}(X_{\{PAR_n\}}) \geq n - c$ for some absolute constant $c \geq 0$, where $X_{\{PAR_n\}} = PAR_n^{-1}(0) \times PAR_n^{-1}(1)$ is equipped with the disagreement-Hamming metric $d((x,y),(x',y')) = \#\{i : x_i \neq x'_i \text{ or } y_i \neq y'_i, i \in \text{disagree}(x,y) \cup \text{disagree}(x',y')\}$. Combined with axiom A1 (Tree-to-Dimension), this yields $D(PAR_n) \geq n - c$, matching the known tight bound $D(PAR_n) = n$. Equivalently, no $\text{AsymDim}_{\text{cover}}$ of $X_{\{PAR_n\}}$ with bounded diameter can have multiplicity smaller than $n - c + 1$.

Rationale (proposer's reasoning):

This sub-conjecture directly stress-tests axiom A1 on the canonical function whose exact De Morgan formula depth (n) is known via Khrapchenko's bound. If A1 is to be a viable route to depth lower bounds, the coarse-geometric invariant $\text{asdim}(X_f)$ must at minimum recover this textbook tight case. Failure ($\text{asdim}(X_{\{PAR_n\}}) = o(n)$) would refute A1 quantitatively; success (asdim grows linearly in n) provides the first nontrivial empirical evidence that the coarse-geometric KW pipeline recovers Khrapchenko-strength bounds without invoking the quadratic measure, and it would license the use of asdim as a substitute invariant in the KRW composition program (axiom A3).

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_6ef88ddb3, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4fa2cb8f0f7e4e28

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For n in $\{2,3,4,5,6\}$, compute LP-based lower bound $L(n)$ on $\text{asdim}(X_{\{\text{PAR}_n\}})$ via ball-packing dual at best scale r in $\{1, \dots, n\}$. Fit $L(n) = a*n + b$ via least squares. Conjecture SUPPORTED iff slope $a \geq 0.8$ AND intercept $b \geq -3$ AND $L(n) \geq n-3$ for at least 4 of 5 values of n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson parity formula depth lower bound - asymptotic dimension Karchmer-Wigderson communication complexity - coarse geometry asymptotic dimension Boolean formula depth

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/0711.0446v1>] Asymptotic Properties of Hilbert Geometry - [<http://arxiv.org/abs/2110.01377v1>] Linearly ordered coarse spaces - [<http://arxiv.org/abs/math/0601744v1>] Coarse geometry and asymptotic dimension - [s2:2410.10189] Toward Better Depth Lower Bounds: Strong Composition of XOR and a Random Function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hamming_distance(x, y):
        return sum(1 for a, b in zip(x, y) if a != b)

    def linear_programming(metric, r):
        n = len(metric)
        c = [-1] * n
```

```

A = []
b = []

for i in range(n):
    row = [0] * n
    for j in range(n):
        if hamming_distance(i, j) <= r:
            row[j] = 1
    A.append(row)
    b.append(1)

# Solve the LP dual using Gaussian elimination
A = [row + [c[i]] for i, row in enumerate(A)]
n = len(A)
m = len(A[0])
pivot_col = 0

for i in range(n):
    if A[i][pivot_col] == 0:
        for j in range(i+1, n):
            if A[j][pivot_col] != 0:
                A[i], A[j] = A[j], A[i]
                break
    else:
        pivot_col += 1
        continue

    for j in range(n):
        if i != j and A[j][pivot_col] != 0:
            factor = -A[j][pivot_col] / A[i][pivot_col]
            A[j] = [A[j][k] + factor * A[i][k] for k in range(m)]

y = [row[-1] for row in A if all(row[k] == 0 for k in range(n))]
return sum(y)

def asdim(X):
    n = len(X)
    metric = [[hamming_distance(i, j) for j in X] for i in X]
    max_multiplicity = 0
    for r in range(1, n+1):
        multiplicity = linear_programming(metric, r)
        if multiplicity > max_multiplicity:
            max_multiplicity = multiplicity
    return max_multiplicity

def generate_PAR_n(n):
    base = [tuple(i) for i in itertools.product([0, 1], repeat=n)]
    X_0 = [x for x in base if sum(x) % 2 == 0]
    X_1 = [x for x in base if sum(x) % 2 != 0]
    return X_0 + X_1

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    X = generate_PAR_n(n)

```

```

    asdim_value = asdim(X)
    results.append(asdim_value)

a, b = polyfit(range(len(results)), results, 1)
mean_asdim = sum(results) / len(results)
std_asdim = math.sqrt(sum((x - mean_asdim) ** 2 for x in results) / len(results))

conjecture_holds = a >= 0.8 and b >= -3
counterexample = "" if conjecture_holds else f"asdim(X_{n_values[-1]}) = {mean_asdim}, expected"

return {
    "metric_name": "asdim",
    "metric_value": mean_asdim,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 30 primes if

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean_asdim = sum(results) / len(results)
    std_asdim = math.sqrt(sum((x - mean_asdim) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= seeds[-1] - 3) / len(results)

    if all(r >= seeds[-1] - 3 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_asdim} std={std_asdim} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_asdim} std={std_asdim} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if r < seeds[-1] - 3)
        print(f"RESULT: FALSIFIED counterexample='asdim(X_{seeds[-1]}) = {results[first_failing_seed]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ec93f4c.py", line 108, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ec93f4c.py", line 83, in run_trial

```

    asdim_value = asdim(X)
    ^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ec93f4c.py", line 67, in asdim
    multiplicity = linear_programming(metric, r)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ec93f4c.py", line 33, in linear_progra
    if hamming_distance(i, j) <= r:
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ec93f4c.py", line 22, in hamming_dista
    return sum(1 for a, b in zip(x, y) if a != b)
               ^^^^^^^^^^

```

TypeError: 'int' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a TypeError ('int' object is not iterable) in hamming_distance before any L(n) values were computed, so the pre-registered support condition cannot be evaluated. | next: Fix the type bug by ensuring elements of X are passed as tuples/sequences (e.g., flatten (x,y) pairs into bit-vectors) before computing Hamming distance, then rerun for n in {2,...,6}.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	6934
2	novelty	claude_max	opus	0	0	3328
3	novelty	claude_max	opus	0	0	10165
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22549
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	192593
6	judge	claude_max	opus	0	0	5505

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241074 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/b0765766a778.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b0765766a778.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b0765766a778.tar.gz` (if generated)